Distributed Image Processing Pipeline

Kafka · Redis · Docker — Project Guide

A single-machine rebuild of a distributed image processing system. Users upload an image and choose a filter; the image is cut into tiles that worker containers process in parallel through Apache Kafka, and Redis holds the shared state, worker heartbeats and a cache of finished images. Everything runs as 16 Docker containers and survives the failure of a Kafka broker, a worker, or the Redis master.

This guide covers

1. What the project does and how its parts fit together
2. The end-to-end workflow of one upload
3. Docker: the image, the containers, networking and start order
4. Kafka: KRaft controller, brokers, topics, consumer groups, offsets
5. Redis: replication, Sentinel failover, data structures, the cache
6. A tour of the code, file by file
7. Fault tolerance and what was tested
8. Design decisions and trade-offs, demo commands, interview Q&A

October 2026

Contents

1. The project at a glance **4**

1.1 Technology . . . 4

1.2 Key numbers . . . 4

2. Architecture **5**

2.1 All 16 containers . . . 6

3. End-to-end workflow: one upload **7**

3.1 Message formats . . . 7

3.2 The three API calls between web and master . . . 7

4. Docker **8**

4.1 One Dockerfile, seven containers . . . 8

4.2 docker-compose.yml: the whole system in one file . . . 8

4.3 Networking: names, IPs and ports . . . 9

4.4 Start order . . . 9

5. Kafka **10**

5.1 Vocabulary, mapped to the project . . . 10

5.2 The Kafka image and how it is configured . . . 10

5.3 KRaft and the controller . . . 11

5.4 Topic layout . . . 11

5.5 setup_kafka.py (the kafka-init container) . . . 12

5.6 Consumer groups and coordinators . . . 12

5.7 Producing (master/tile_publisher.py) . . . 12

5.8 Consuming (worker/worker.py) . . . 13

5.9 Three different heartbeats . . . 13

6. Redis **14**

6.1 Six containers . . . 14

6.2 Replication . . . 14

6.3 Sentinel failover (measured in testing) . . . 14

6.4 Why three Sentinels . . . 14

6.5 Configuration . . . 15

6.6 Connecting through Sentinel (common/redis_store.py) . . . 15

6.7 Data structures . . . 15

6.8 Counting tiles safely: record_tile() . . . 15

7. Code tour **16**

7.1 The tile halo (common/imaging.py) . . . 16

7.2 The dashboard . . . 16

8. Fault tolerance and testing **17**

8.1 Kafka broker failure, step by step 17

8.2 A real bug found during testing 17

9. Design decisions and trade-offs **18**

9.1 Compared with the original project 18

10. Running it and demo commands **19**

11. Interview questions and answers **20**

Appendix A. Specifications **22**

Appendix B. Glossary **22**

1. The project at a glance

What it does. A user opens a website, uploads an image (at least 1024×1024) and picks a filter: **Gaussian Blur** or **Black & White**. The master service cuts the image into 512×512 tiles and puts one message per tile on a Kafka topic. Four worker containers read the tiles in parallel, apply the filter with OpenCV and send the processed tiles back through a second topic. The master collects them, stitches the image back together and stores it in Redis. If the same image is uploaded again with the same filter, it is served straight from the Redis cache in about 2 ms.

Why it is interesting. It shows parallel processing (partitions and consumer groups), fault tolerance at every layer (Kafka replication and leader election, consumer-group rebalancing with committed offsets, Redis replication with Sentinel failover), safe shared state (atomic Redis operations and de-duplication) and caching, all reproducible on a laptop with one command.

1.1 Technology

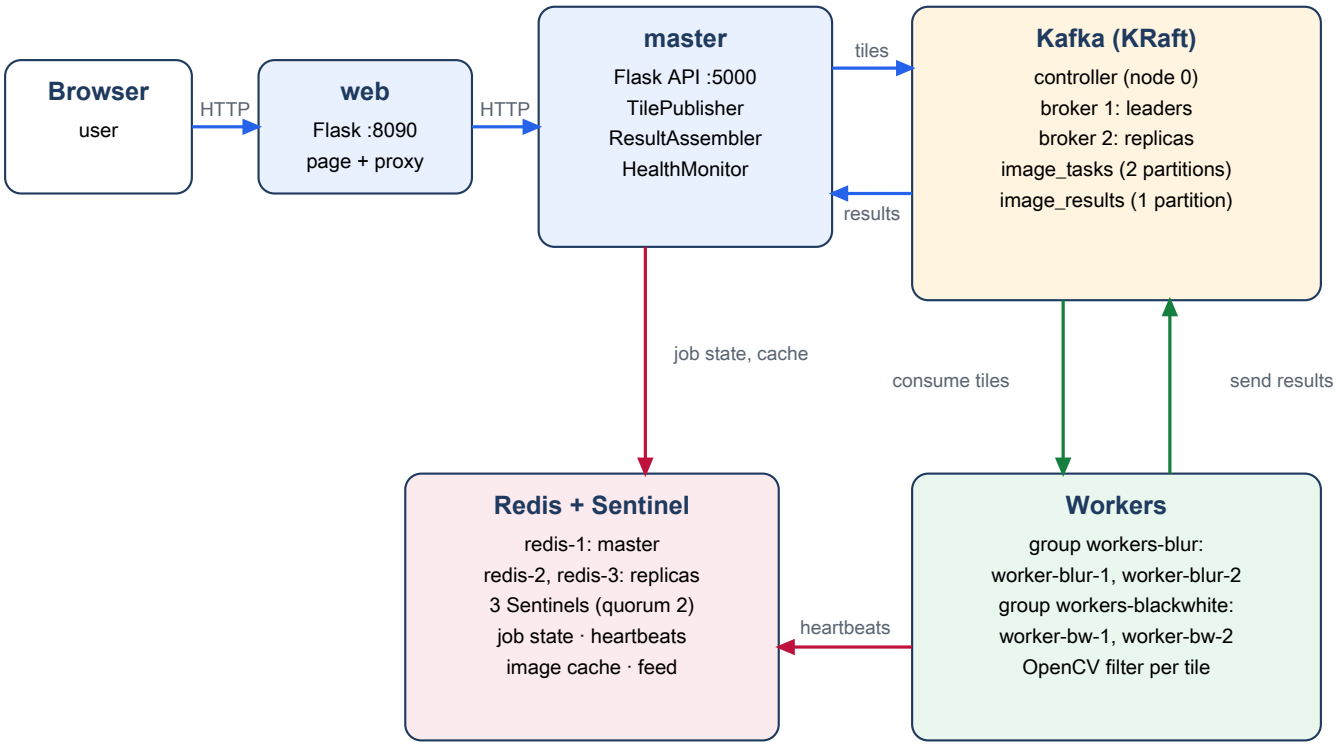
Area	Technology	Used for
Language	Python 3.12	All of our code
Messaging	Apache Kafka 4.1 (KRaft mode)	Carrying tiles to workers and results back
Kafka client	confluent-kafka 2.15	Producer, Consumer and AdminClient in Python
Shared state / cache	Redis 7.4 + Sentinel	Job progress, heartbeats, image cache, feed
Redis client	redis-py 8.1	Connecting through Sentinel, with retries
Image processing	OpenCV 4.13, NumPy	Decode, blur / grayscale, encode, reassemble
Web	Flask 3.1 + Waitress	Website and the master's HTTP API
Packaging	Docker, Docker Compose	One image for our code, 16 containers in total

1.2 Key numbers

Item	Value
Containers	16 (3 Kafka, 1 Kafka setup, 3 Redis, 3 Sentinel, 4 workers, master, web)
Kafka topics	image_tasks (2 partitions), image_results (1 partition), replication factor 2
Tile size / minimum image	512 × 512 px tiles, plus a 25 px halo / 1024 × 1024 px
Example job	2048 × 2048 image = 16 tiles, about 5–7 s (includes a 0.5 s demo delay per tile)
Cache hit	about 2 ms (no Kafka, no workers)
Failover times (measured)	Redis master about 6 s · worker about 10 s · Kafka broker about 10–15 s

2. Architecture

Five kinds of components. The web container is what the user talks to; the master orchestrates; Kafka moves the work; the workers do the image processing; Redis is the shared memory everyone can read.



Component	Role	Analogy
web	Serves the page and forwards the user's requests to the master over HTTP. Knows nothing about Kafka or Redis.	Front desk
master	Checks the cache, splits images into tiles, publishes them to Kafka, collects results, reassembles and caches the image, and monitors worker health.	Head chef
Kafka	Durable message log. Topic *image_tasks*: master → workers. Topic *image_results*: workers → master.	Order tickets on a rail
workers	Read tiles, apply the filter, send the processed tile back, commit their progress.	Line cooks
Redis	Job progress, worker heartbeats, the cache of finished images, the feed and event log.	Shared whiteboard

2.1 All 16 containers

Container	Image	What runs inside	Lifetime
kafka-controller	apache/kafka:4.1.0	Kafka as KRaft controller (node 0): cluster metadata, leader elections	always
kafka-broker-1	apache/kafka:4.1.0	Kafka broker (node 1): preferred leader of every partition	always
kafka-broker-2	apache/kafka:4.1.0	Kafka broker (node 2): holds the follower replicas	always
kafka-init	image-pipeline (ours)	kafka_setup/setup_kafka.py: creates topics, then exits	runs once, exit 0
redis-1	redis:7.4-alpine	Redis server, starts as master	always
redis-2, redis-3	redis:7.4-alpine	Redis servers, replicas of redis-1	always
redis-sentinel-1..3	redis:7.4-alpine	Sentinels: watch the master, run failover	always
worker-blur-1, -2	image-pipeline (ours)	worker/worker.py with WORKER_FILTER=blur	always
worker-bw-1, -2	image-pipeline (ours)	worker/worker.py with WORKER_FILTER=blackwhite	always
master	image-pipeline (ours)	master/app.py (+ producer, consumer, monitor threads)	always
web	image-pipeline (ours)	web/app.py	always

Only two containers are reachable from the host machine: **web** (localhost:8090) and **master** (localhost:5050, for testing). Kafka and Redis are internal to the Docker network.

3. End-to-end workflow: one upload

Example: a user uploads **cat.jpg** (2048×2048) with filter **blur**. It gets job id **a3f9c1** and becomes 16 tiles.

1. **Browser → web.** The page posts the file and filter to the web container.
2. **web → master.** The web app forwards the request to *http://master:5000/api/jobs*.
3. **Cache check.** The master computes the SHA-256 of the file and looks for *cache:<sha>:blur* in Redis. **Hit:** the job is created as done and the user gets the image immediately; steps 4–9 are skipped.
4. **Split.** The image is cut into 512×512 tiles, each padded with 25 px from its neighbours (the halo), so the blur has no seams.
5. **Job state → Redis.** Hash *job:a3f9c1* = {status: processing, total_tiles: 16, processed_tiles: 0, width, height, filter, image_hash}; job id added to the feed.
6. **Tiles → Kafka.** 16 messages to *image_tasks*: key *a3f9c1:0 … a3f9c1:15*, value = tile JPEG bytes, headers = filter and crop box. The client library picks the partition from the key. The HTTP response is sent only after Kafka acknowledged every tile (acks=all).
7. **Workers.** Each worker polls its partition, decodes the tile, applies the filter, cuts off the halo, sends the result to *image_results* (same key), waits for the acknowledgement and only then commits its offset.
8. **Master collects.** A background thread polls *image_results*. For each tile: ZADD NX *job:a3f9c1:done* (ignore duplicates) → save bytes in *job:a3f9c1:tiles* → HINCRBY processed_tiles.
9. **Reassemble.** The call that brings processed_tiles to 16 rebuilds the image, stores it as *cache:<sha>:blur* (24 h) and marks the job done, in one Redis transaction.
10. **User gets it.** The page polls every 1.5 s; once the job is done it downloads the image.

3.1 Message formats

Topic	Key	Value	Headers
image_tasks	job_id:tile_index (e.g. a3f9c1:5)	Padded tile, JPEG bytes	filter (blur / blackwhite), crop (left, top, width, height of the core inside the padding)
image_results	same key	Processed core tile, JPEG bytes	worker (who processed it), filter, src (source partition:offset), proc_ms

Why the key is job_id:tile_index. The partition is chosen as hash(key) % 2, so a unique key per tile spreads one image's tiles over both partitions (e.g. P0: 9, P1: 7) and both workers in a group share the image. If the key were only the job id, every tile of an image would land in one partition and only one worker would process it.

3.2 The three API calls between web and master

Call	Purpose
POST /api/jobs	Upload an image + filter. Returns the job id (200 on cache hit, 202 when queued).
GET /api/jobs/<id>	Job status: processed/total tiles, which worker did which tile, partition split.
GET /api/jobs/<id>/result	The finished image (JPEG), read from the Redis cache.

Helper endpoints for the page: /api/feed (newest jobs), /api/cluster (dashboard data), /api/jobs/<id>/preview (thumbnail), /healthz.

4. Docker

Docker runs each program in its own **container**: an isolated box with its own files, network address and processes. Containers share the host's kernel, so they start in about a second and are much lighter than virtual machines. That is what makes 16 services on one laptop practical, and identical on any machine.

Term	What it is	Analogy
Dockerfile	Text file with build instructions	Source code of a class
Image	The built, read-only package: OS + Python + libraries + code	The compiled class
Container	A running instance of an image	An object

4.1 One Dockerfile, seven containers

Kafka and Redis use their official images from Docker Hub, so the only Dockerfile is for our own code. All Python services need the same libraries, so one image (*image-pipeline:latest*) holds all the code, and docker-compose starts seven containers from it, each running a different file.

```
FROM python:3.12-slim                          # small Debian + Python 3.12
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1                         # logs appear immediately in `docker logs`
WORKDIR /app
COPY requirements.txt .                        # dependencies first ...
RUN pip install --no-cache-dir -r requirements.txt
COPY common ./common                           # ... code last
COPY kafka_setup ./kafka_setup
COPY master ./master
COPY worker ./worker
COPY web ./web
COPY tools ./tools
```

- **Layers and caching.** Each instruction creates a layer. On a rebuild, unchanged layers are reused (CACHED); a changed layer and every layer after it are rebuilt. Copying requirements.txt and installing before copying the code means a code change rebuilds in seconds instead of reinstalling OpenCV.
- **RUN vs command.** RUN executes once at build time; the program a container runs is set per container in docker-compose.yml (there is no CMD in the Dockerfile).
- **Containers run the copy inside the image**, so after editing code you must rebuild (docker compose build) and restart.
- **.dockerignore** keeps .git, caches, docs and sample images out of the build.

4.2 docker-compose.yml: the whole system in one file

The Dockerfile says what is inside one box; docker-compose.yml says which boxes run and how they are set up. It decides:

1. Which containers exist (16) and which image each uses.
2. What each container runs (*command*): worker, master, web, setup script.
3. Each container's settings (*environment*): node ids, worker names, which filter a worker handles.
4. How they talk: one shared network where containers reach each other by name.
5. What the host can reach (*ports*): web 8090→8080 and master 5050→5000 only.
6. Start order (*depends_on*): Kafka before topics, topics before workers, master before web.
7. Config files (*volumes*): the Redis and Sentinel config files are mounted read-only.

The four workers run the same image and the same command; only their environment variables (WORKER_ID, WORKER_FILTER) differ. YAML anchors (*&name* / *<<: *name*) avoid repeating shared settings.

4.3 Networking: names, IPs and ports

- Compose creates one private network, *image-pipeline_default*. Every container gets its own IP and Docker's built-in DNS resolves container names to those IPs. Code always uses names (kafka-broker-1:9092, http://master:5000), never IPs, because IPs change on restart.
- Ports are fixed by configuration (Kafka 9092, controller 9093, Redis 6379, Sentinel 26379, master 5000, web 8080). Because each container has its own IP, both brokers can use port 9092 without a clash. On the host there is only one IP, which is why the website is published on 8090 (8080 was already taken on the host).

4.4 Start order

Condition	Waits until	Used for
service_started	the container has started (not necessarily ready)	brokers → controller, Python → Sentinels
service_healthy	its healthcheck passes	replicas & Sentinels → redis-1, web → master
service_completed_successfully	it exited with code 0	every Python service → kafka-init

Started is not the same as ready, so the Python code also retries: setup_kafka.py waits until both brokers have registered, and every service waits until Sentinel knows a Redis master that answers PING.

5. Kafka

Kafka is a durable, replicated message log. Producers append messages to topics; consumers read them at their own pace. Messages are not deleted when read (only after the retention time), and each consumer group remembers how far it has read. Compared with sending tiles directly, Kafka gives decoupling, buffering, automatic work distribution and crash recovery.

5.1 Vocabulary, mapped to the project

Term	Meaning	In this project
Topic	Named stream of messages	image_tasks, image_results
Partition	Ordered, append-only log; unit of parallelism; numbered per topic from 0	image_tasks P0, P1; image_results P0
Offset	Position of a message in a partition, assigned by the leader	image_tasks[P1] offset 42
Broker	Server that stores partitions and serves clients	kafka-broker-1, kafka-broker-2
Replication factor	Number of copies of each partition	2
Leader / follower	Leader handles all reads and writes; followers copy it	leader on broker 1, follower on broker 2
ISR	In-sync replicas: copies fully caught up with the leader	normally [1, 2]; [2] if broker 1 dies
Controller	Cluster brain: metadata, topic creation, leader election	kafka-controller (node 0)
Consumer group	Consumers sharing a group.id; each partition goes to one member	workers-blur, workers-blackwhite, master-assemblers
Group coordinator	Broker that manages one group: members, assignment, committed offsets	one broker per group (Section 5.6)
Committed offset	Where a group will continue reading a partition	lets a worker resume after a crash

5.2 The Kafka image and how it is configured

The three Kafka containers use the official **apache/kafka:4.1.0** image (Java). Kafka 4 runs only in KRaft mode; ZooKeeper support was removed. We never write a Kafka config file: the image's start script turns every *KAFKA_** environment variable into a line in */opt/kafka/config/server.properties* (drop KAFKA_, lowercase, _ → .), formats the storage with CLUSTER_ID on first start, then starts Kafka.

```
KAFKA_NODE_ID: 1                          ->  node.id=1
KAFKA_PROCESS_ROLES: broker               ->  process.roles=broker
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka-broker-1:9092
                                          ->  advertised.listeners=PLAINTEXT://kafka-broker-1:9092
```

Setting	Value	Why
CLUSTER_ID	shared by all 3 nodes	nodes refuse to join a cluster with a different id
CONTROLLER_QUORUM_VOTERS	0@kafka-controller:9093	who the controller is; brokers use it to find it
PROCESS_ROLES	controller / broker	which role the node plays
LISTENERS / ADVERTISED_LISTENERS	:9092 / kafka-broker-N:9092	where it binds / the address handed to clients in metadata (must be reachable, so the container name)
AUTO_CREATE_TOPICS_ENABLE	false	topics exist only if kafka-init created them
DEFAULT_REPLICATION_FACTOR / MIN_INSYNC_REPLICAS	2 / 1	two copies; writes continue with one broker down
OFFSETS_TOPIC_NUM_PARTITIONS / RF	2 / 2	two possible coordinators; committed offsets survive a broker failure
LEADER_IMBALANCE_CHECK_INTERVAL_SECONDS (controller)	30	give leadership back to the preferred leader after recovery

5.3 KRaft and the controller

KRaft (Kafka + Raft) is Kafka's built-in way of storing cluster metadata. The metadata is kept as an ordered log, *__cluster_metadata*, owned by the controller; brokers follow that log and keep a live copy. One authority and one ordered log mean every broker agrees on who leads what (no split brain). **There is no KRaft code in this project**: it is inside the Kafka image. We only enable it (PROCESS_ROLES=controller and the voters list), send it requests (create topics) and read the metadata it maintains.

Metadata recorded	Values in this project
Controller	node 0 at kafka-controller:9093
Brokers	broker 1 (kafka-broker-1:9092), broker 2 (kafka-broker-2:9092), each with a broker epoch; fenced or not
Topics + configs	image_tasks, image_results, __consumer_offsets; min.insync.replicas=1, retention.ms=3600000
Partitions	for each of the 5 partitions: replicas, leader, ISR, leader epoch
Changes	every leader / ISR change is appended as a new record

Controller job	When it happens here
Register brokers and track liveness	brokers heartbeat every 2 s; silent for 9 s → fenced (marked dead)
Create topics, place partitions, pick leaders	kafka-init's CreateTopics is forwarded by a broker to the controller
Elect a new partition leader	a broker dies → first replica in the list that is alive and in the ISR
Record ISR changes	followers falling behind or catching up
Restore preferred leaders	every 30 s, after the failed broker is back in sync

Two different elections. Raft elects which *controller* is active (a majority vote among controllers; trivial here with one controller). Partition leader election is the active controller *choosing* a broker from the ISR; there is no vote. The controller stores no messages and does not manage consumer groups.

5.4 Topic layout

Defined in *common/config.py* and created by *setup_kafka.py*. Each inner list is one partition; its length is the replication factor; the first broker is the preferred leader.

```
TOPIC_LAYOUT = {
    "image_tasks":   [[1, 2], [1, 2]],   # 2 partitions, RF 2, leader = broker 1
    "image_results": [[1, 2]],           # 1 partition,  RF 2, leader = broker 1
}
```

Partition	Replicas	Leader	ISR
image_tasks-0	[1, 2]	1	[1, 2]
image_tasks-1	[1, 2]	1	[1, 2]
image_results-0	[1, 2]	1	[1, 2]
__consumer_offsets-0 / -1	placed by Kafka	one per broker	[1, 2]

Leaders were pinned on broker 1 so failover is easy to demonstrate. Kafka's default would spread leaders across brokers (e.g. *[[1, 2], [2, 1]]*) to share the load. With RF 2 on 2 brokers, each broker holds a copy of every partition either way.

5.5 setup_kafka.py (the kafka-init container)

Step	Function	What it does
1	wait_for_brokers()	Polls metadata until brokers 1 and 2 are registered (started ≠ ready).
2	show_cluster()	Prints the cluster id.
3	create_topics()	Sends NewTopic(name, partitions, replica_assignment, configs) via AdminClient.create_topics(). TOPIC_ALREADY_EXISTS is ignored, so re-running is safe.
4	wait_for_partitions()	Creation is asynchronous: waits until every partition has a leader, prints leader / replicas / ISR.
5	show_coordinators()	Looks up each group's coordinator (this also creates __consumer_offsets) and checks it against the formula.
–	exit code	0 → Compose starts workers and master. 1 → nothing else starts.

```
NewTopic("image_tasks", num_partitions=2, replica_assignment=[[1, 2], [1, 2]],
         config={"min.insync.replicas": "1", "retention.ms": "3600000"})
admin.create_topics(new_topics)     # script -> broker -> forwarded to the controller -> recorded
```

5.6 Consumer groups and coordinators

Group	Members	Reads
workers-blur	worker-blur-1 (P0), worker-blur-2 (P1)	image_tasks; processes blur tiles, skips others
workers-blackwhite	worker-bw-1 (P0), worker-bw-2 (P1)	image_tasks; processes black & white tiles, skips others
master-assemblers	the master's ResultAssembler thread	image_results

Different groups each receive every message, with independent offsets; inside a group each partition goes to exactly one member. That is why the two worker groups are split by filter: otherwise every tile would be processed twice.

The group coordinator is the broker that manages a group: it tracks members through their heartbeats, hands out partitions, stores committed offsets in *__consumer_offsets*, and rebalances when a member joins or leaves. It cannot be assigned directly:

```
partition   = abs(javaHashCode(group.id)) % offsets.topic.num.partitions     (= 2 here)
coordinator = the broker leading that partition of __consumer_offsets

workers-blur       -> P1        workers-blackwhite -> P0        master-assemblers -> P0
```

The group names were chosen so the two worker groups hash to different partitions, which have leaders on different brokers. Which broker leads P0 vs P1 is decided by Kafka on a fresh cluster, so the pair can come out swapped; they are always on different brokers. If a coordinator's broker dies, the surviving broker becomes leader of that __consumer_offsets partition and takes over as coordinator, with all committed offsets.

5.7 Producing (master/tile_publisher.py)

```
Producer({"bootstrap.servers": "kafka-broker-1:9092,kafka-broker-2:9092",
          "acks": "all",                    # leader waits for every in-sync replica
          "enable.idempotence": True,       # network retries cannot create duplicates
          "partitioner": "murmur2_random"}) # Java-compatible hash(key) % partitions
producer.produce("image_tasks", key=f"{job_id}:{tile.index}", value=tile.data,
                 headers=[("filter", flt), ("crop", "...")], on_delivery=on_delivery)
producer.flush(30)                          # HTTP response only after every tile is acknowledged
```

- **Bootstrap servers**: the client contacts one, downloads the cluster map, then talks directly to each partition's leader.

- **acks=all + min.insync.replicas=1**: a write is confirmed once every in-sync copy has it; with broker 1 down the ISR is [2], which still meets the minimum, so writes continue. Setting 2 would favour durability over availability.
- The delivery callback logs the partition and the offset the leader assigned.

5.8 Consuming (worker/worker.py)

```
Consumer({"group.id": "workers-blur", "enable.auto.commit": False,
          "auto.offset.reset": "earliest",
          "partition.assignment.strategy": "cooperative-sticky",
          "session.timeout.ms": 10000, "heartbeat.interval.ms": 3000})

msg = consumer.poll(1.0)                                  # read a tile
result = encode(crop_core(apply_filter(decode(msg.value()), flt), crop))
producer.produce("image_results", key=msg.key(), value=result, headers=[...])
producer.flush(30)                                        # wait for the acknowledgement
consumer.commit(message=msg, asynchronous=False)          # only now: offset + 1 = next to read
```

- **Commit after the result is acknowledged** → at-least-once delivery. If the worker crashes between sending and committing, the next owner reprocesses the tile; the master drops the duplicate.
- If the result is not acknowledged, the worker does not commit; it seeks back and retries.
- **Rebalance callbacks**: on_assign (logs the committed offset it resumes from), on_revoke, on_lost (kicked out after a session timeout).
- **cooperative-sticky**: only partitions that need to move are revoked; the rest keep working.
- **SIGTERM** (docker stop) → consumer.close() sends LeaveGroup → rebalance within about 3 s instead of waiting 10 s.
- A stale commit from a frozen (zombie) worker is rejected by the coordinator (UNKNOWN_MEMBER_ID).

5.9 Three different heartbeats

Heartbeat	From → to	Interval / timeout	Purpose	Our code?
Broker heartbeat	broker → controller	2 s / 9 s	Is the broker alive? If not, elect new partition leaders	No, Kafka internal (defaults)
Consumer heartbeat	worker → group coordinator	3 s / 10 s	Is the worker alive? If not, rebalance its partitions	Settings only (config.py)
Redis heartbeat	worker → Redis key with TTL	3 s / 10 s	Show the user that a worker died	Yes: worker.py, health_monitor.py

6. Redis

Redis is an in-memory key-value database: think of a Python dictionary on a server that every container can use. Reads and writes take well under a millisecond, and Redis executes commands **one at a time**, so every single command is atomic. Each container has its own memory, so the services need one shared place; Redis also survives a master restart and offers expiring keys and atomic counters.

Job	What is stored	Written by	Read by
Job progress	status, total and processed tiles, size, filter	master	master, web
Worker heartbeats	a key per worker that expires after 10 s	workers	master's monitor
Cache	finished images, keyed by SHA-256 + filter (24 h)	master	master
Feed and events	recent jobs, dashboard event log	master, workers	web

6.1 Six containers

Container	Program	Role
redis-1	redis-server	Master: all writes go here
redis-2, redis-3	redis-server --replicaof redis-1 6379	Replicas: live, read-only copies
redis-sentinel-1..3	redis-sentinel	Watchers: detect a dead master, promote a replica, tell clients who the master is

6.2 Replication

A replica connects to the master, receives a full snapshot, then receives every write as it happens. Replicas are read-only. Replication is **asynchronous**: the master answers before the replicas have the write, so a failover can lose the last few milliseconds of writes. Replication alone does **not** fail over: if the master dies, replicas just keep trying to reconnect. That is Sentinel's job.

6.3 Sentinel failover (measured in testing)

Time	Step	Meaning
0 s	redis-1 killed	Every Sentinel keeps pinging it, no reply
+5 s	+sdown	One Sentinel: “I think it is down” (down-after-milliseconds 5000)
+5 s	+odown #quorum 3/2	At least 2 Sentinels agree → officially down
+5–6 s	vote, promote	Sentinels vote for one leader; it promotes the most up-to-date replica (REPLICAOF NO ONE) and repoints the other
+6 s	+switch-master … redis-2	All Sentinels now answer “redis-2”; clients reconnect there
later	+convert-to-slave	redis-1 comes back as a replica of the new master (it does not take its job back)

Sentinels discover each other and the replicas by exchanging hello messages through the master (gossip), use a **quorum** to agree the master is down, and a **majority vote** to pick which Sentinel performs the failover. All of this is inside the Redis image; we only configure it.

6.4 Why three Sentinels

- One Sentinel is a single point of failure: if it is down, nobody fails over.
- A Sentinel cannot tell “master is dead” from “my network link is broken”. Alone, it could promote a replica while the old master is still taking writes: two masters, lost data (split brain). With 3 Sentinels and quorum 2, one confused Sentinel cannot trigger a failover.
- 3 is the smallest number where a majority (2) survives losing one. With 2, losing one leaves no majority.
- The same rule applies to KRaft: one controller is a demo shortcut; production uses 3.

6.5 Configuration

```
# infra/redis/redis.conf  (all three servers)
appendonly yes             # write every change to disk; reload it on restart
appendfsync everysec
maxmemory 512mb
maxmemory-policy volatile-lru   # when full, evict least-recently-used keys that have a TTL
replica-read-only yes

# infra/redis/sentinel.conf
sentinel monitor mymaster redis-1 6379 2        # name, initial master, quorum
sentinel down-after-milliseconds mymaster 5000
sentinel failover-timeout mymaster 20000
```

6.6 Connecting through Sentinel (common/redis_store.py)

```
def connect(decode=True):
    return Sentinel(REDIS_SENTINELS).master_for(
        "mymaster",                         # never "redis-1": always ask who the master is
        decode_responses=decode,            # True: text (metadata); False: bytes (images)
        socket_timeout=3.0,
        retry=Retry(ExponentialBackoff(cap=1.0, base=0.1), retries=25))   # ~20 s of retries
```

When the master dies, the next command fails; the client waits and retries, asks Sentinel again on reconnect, and reaches the promoted replica. No code change and no restart. *wait_for_redis()* blocks at startup until Sentinel knows a master that answers PING.

6.7 Data structures

Type	Our keys (example)	Feature relied on
String	cache:9b2e…:blur → image bytes (TTL 24 h) worker:worker-blur-1:heartbeat → JSON (TTL 10 s) preview:9b2e… → thumbnail	TTL: automatic expiry. A worker resets its heartbeat to 10 s every 3 s; if it dies the key disappears and the master marks it dead.
Hash	job:a3f9c1 → {status, total_tiles, processed_tiles, …} job:a3f9c1:tiles → {tile index: bytes} job:a3f9c1:workers, worker:<id>:info, stats	Update one field atomically with HINCRBY.
Sorted set	job:a3f9c1:done → tile indexes (score = arrival time) jobs:feed → job ids (score = upload time) workers:last_seen → worker ids	Unique members (ZADD NX de-duplicates) and kept sorted by score (ZREVRANGE = newest first).
List	events → newest-first dashboard log	LPUSH + LTRIM keeps the newest 200.

6.8 Counting tiles safely: record_tile()

```
is_new = r.zadd(f"job:{id}:done", {tile_index: now}, nx=True)   # 1 = new, 0 = already had it
if not is_new:
    return                                                      # duplicate -> ignore
r.hset(f"job:{id}:tiles", tile_index, tile_bytes)               # save bytes BEFORE counting
processed = r.hincrby(f"job:{id}", "processed_tiles", 1)        # atomic +1, returns new value
if processed == total: assemble()                               # exactly one caller sees this
```

- **Duplicates.** Kafka is at-least-once, so a tile can arrive twice (worker crashed after sending, before committing). ZADD NX returns 0 for a tile already recorded, so it is not counted again. Without it the count could reach 16 with one tile missing.
- **Lost updates.** A read-then-write from two places could both read 5 and both write 6. HINCRBY does the increment inside Redis in one step, and each caller gets a different value back, so exactly one caller sees processed == total and reassembles.
- **Why in Redis, not a Python set?** The counter lives in Redis and survives a master restart; the record of counted tiles must live next to it, or re-read results after a restart would be counted twice.

7. Code tour

File	Responsibility	Key parts
Dockerfile	Builds the one Python image	layer order: requirements first, code last
docker-compose.yml	Defines all 16 containers	Kafka env settings, Redis commands, worker env vars, ports, depends_on
infra/redis/*.conf	Redis server and Sentinel settings	persistence, eviction, quorum, timeouts
common/config.py	Every setting in one place	TOPIC_LAYOUT, group names, tile size, timeouts, TTLs
common/kafka_internals.py	Kafka helpers	group → coordinator formula; metadata → table
common/redis_store.py	Redis toolbox	connect() via Sentinel, key names, record_tile(), push_event()
common/imaging.py	Pure image logic (OpenCV)	split_into_tiles() with halo, apply_filter(), crop_core(), assemble()
kafka_setup/setup_kafka.py	One-time Kafka setup	wait for brokers, create topics, print leaders and coordinators
master/app.py	HTTP API and wiring	POST /api/jobs (cache check, split, job state, publish), status, result, feed, cluster
master/tile_publisher.py	Producer role	acks=all, idempotence, keys, headers, delivery callback, flush
master/result_assembler.py	Consumer role (thread)	poll results, record_tile, reassemble, cache write in a MULTI/EXEC transaction, manual commit
master/health_monitor.py	Monitor (thread, every 3 s)	Redis heartbeat check, Kafka leaders/ISR/offsets/lag, Sentinel's current master
worker/worker.py	Worker	consumer config, rebalance callbacks, filter, produce → flush → commit, heartbeat thread, graceful shutdown
web/app.py + static/	Website	serves the page and proxies API calls; the page polls every 1.5 s
tools/make_test_image.py	Test data	generates the images in samples/

7.1 The tile halo (common/imaging.py)

Blurring each tile on its own leaves visible seams on the tile grid, because pixels at a tile edge have no neighbours to average. Each tile is therefore sent with 25 px (the 51×51 kernel radius) borrowed from its neighbours; the worker blurs the padded tile and crops the halo off using the *crop* header. Every pixel is blurred with its real neighbours, so the reassembled image has no seams.

7.2 The dashboard

The page shows the feed (original preview with a grid coloured by the worker that processed each tile, then the result; click to compare before/after) and a live cluster panel: workers and their heartbeat TTL, Kafka partition leaders and ISR, each consumer group's coordinator, committed offset, end offset and lag, the current Redis master, and the event log.

8. Fault tolerance and testing

Failure	Detected by	How	Recovery	Measured
Kafka broker 1 dies	KRaft controller	broker heartbeats stop (9 s)	Controller fences it and makes broker 2 leader of every partition (ISR → [2]); clients refresh metadata and continue. On return: catches up, rejoins ISR, preferred leader restored within 30 s.	~10–15 s
Worker crashes (docker kill)	Group coordinator + Redis monitor	consumer heartbeats stop (10 s) / heartbeat key expires (10 s)	Rebalance: the other worker in the group takes the partition and resumes from the committed offset.	~10 s
Worker stops (docker stop)	Group coordinator	LeaveGroup on SIGTERM	Immediate rebalance; monitor shows “stopped gracefully”.	~3 s
Worker frozen (docker pause)	Group coordinator	session timeout	Partition reassigned; when it wakes, its commit is rejected (UNKNOWN_MEMBER_ID) and its duplicate tile is ignored by ZADD NX.	~10 s
Redis master dies	3 Sentinels	pings fail (5 s), quorum 2/3	A replica is promoted; clients find it through Sentinel; the old master rejoins as a replica.	~6 s

8.1 Kafka broker failure, step by step

1. Brokers heartbeat to the controller every 2 s. Broker 1 is killed; its heartbeats stop.
2. After 9 s the controller fences broker 1.
3. For every partition broker 1 led, it picks the first replica that is alive and in the ISR (broker 2), removes broker 1 from the ISR, bumps the leader epoch, and records the change.
4. Broker 2 reads the change and becomes leader; it also becomes coordinator for any group whose __consumer_offsets partition it now leads.
5. Clients get connection errors, refresh metadata from broker 2 and continue. With min.insync.replicas=1, acks=all writes still succeed.
6. Broker 1 restarts: re-registers with a new epoch, copies what it missed, rejoins the ISR; within 30 s the controller makes it leader again because it is first in the replica list.

Only in-sync replicas can become leader, because with acks=all they are guaranteed to hold every acknowledged message (unclean.leader.election.enable=false). The leader epoch rejects a stale former leader that wakes up still thinking it is in charge.

8.2 A real bug found during testing

The first version of the health monitor used *AdminClient.describe_consumer_groups*. When broker 1, a group coordinator, was killed, librdkafka 2.15 hit an internal assertion and crashed the whole master process (exit code 133). The crash was reproduced in isolation: the coordinator-based admin calls crashed, while list_topics and list_offsets survived. The monitor was rewritten to read committed offsets with a plain Consumer per group (which never subscribes, so it never joins the group) and to derive the coordinator from metadata. The master then survived the same broker failure.

9. Design decisions and trade-offs

Decision	Why	Trade-off / alternative
Two consumer groups on one topic, one per filter	Demonstrates fan-out, independent offsets and rebalances, and one coordinator per broker.	Each group reads every tile and skips about half; filters cannot share idle workers. Simpler and more efficient: one group where any worker applies the filter from the header (partitions = workers), or one topic per filter if filters need different scaling.
Leaders pinned on broker 1	Clear failover demo.	Default spreading balances load across brokers.
min.insync.replicas = 1	Writes continue with one broker down.	2 would never accept a write held by only one copy, but would block writes during a failure.
One KRaft controller	Light on a laptop.	Single point of failure for metadata changes; production uses 3.
3 Sentinels, quorum 2	Survives one Sentinel failure and avoids split brain.	Could co-locate each Sentinel with a Redis server (3 containers).
Commit offsets after the result is acknowledged	Never lose a tile (at-least-once).	Duplicates possible; handled by ZADD NX. Exactly-once would need Kafka transactions.
SHA-256 + filter as cache key	Same file + same filter → same key; collisions practically impossible.	CRC32 is 32 bits and collides (~50% chance among ~77k images). Exact bytes only; near-duplicates would need a perceptual hash.
Tiles stored in Redis, not on disk	Master can restart mid-job; containers should not rely on local disk.	Uses Redis memory (bounded by maxmemory and TTLs).
Separate web container	Clean split between UI and orchestration.	One extra container and an HTTP hop.
Auto topic creation off	A typo cannot create junk topics.	Topics must be created explicitly (kafka-init).

9.1 Compared with the original project

	Original (3 machines over ZeroTier)	This project (one machine, Docker)
Kafka	1 broker, ZooKeeper, RF 1	KRaft controller + 2 brokers, RF 2
Redis	1 server	Master + 2 replicas + 3 Sentinels
Website	Part of the master	Separate web container
Repeat uploads	Processed again	Served from the Redis cache in ~2 ms
Tile edges	Visible seams after blur	25 px halo, no seams
Processed tiles	Saved on the master's disk	Stored in Redis
Offset commits	Automatic	Manual, after the result is acknowledged
Duplicate tiles	Counted twice	Ignored (ZADD NX)

10. Running it and demo commands

All commands run from the project folder. The full list with expected output is in *docs/demo-commands.md*.

Start and stop

```
docker compose build             # Dockerfile -> image-pipeline:latest
docker compose up -d             # create + start all 16 containers in order
docker compose ps -a             # 15 Up, kafka-init Exited (0)
docker compose logs -f master    # follow a container's logs
docker compose stop              # stop, keep data      |  docker compose down = stop and delete
```

Inspect Kafka and Redis

```
docker compose logs kafka-init                                    # topics, leaders, coordinators
B="docker exec kafka-broker-1 /opt/kafka/bin"
$B/kafka-metadata-quorum.sh --bootstrap-server kafka-broker-1:9092 describe --status   # LeaderId: 0
$B/kafka-topics.sh --bootstrap-server kafka-broker-1:9092 --describe --topic image_tasks
$B/kafka-consumer-groups.sh --bootstrap-server kafka-broker-1:9092 --describe --group workers-blur
docker exec redis-1 redis-cli INFO replication                    # role:master, connected_slaves:2
docker exec redis-sentinel-1 redis-cli -p 26379 SENTINEL get-master-addr-by-name mymaster
docker exec -it redis-1 redis-cli MONITOR                         # every command, live
```

Failover demos

```
# Kafka broker
docker compose stop kafka-broker-1     # wait 15 s -> describe shows Leader: 2, Isr: 2
docker compose start kafka-broker-1    # wait 45 s -> Leader: 1, Isr: 2,1

# Worker (start a 48-tile job first: samples/large_4096x3072.jpg with blur)
docker kill worker-blur-1              # wait 15 s -> both partitions owned by worker-blur-2
docker compose logs worker-blur-2 | grep rebalance    # resuming from committed offset N
docker start worker-blur-1             # partitions split again

# Redis master
docker compose stop redis-1            # wait 10 s -> Sentinel reports redis-2 (or redis-3)
docker exec redis-2 redis-cli KEYS "cache:*"          # cache survived
docker compose start redis-1           # returns as a replica (role:slave)
```

The website at **http://localhost:8090** shows each failure live: a red dot for a dead worker, partition leaders changing, and the new Redis master.

11. Interview questions and answers

Q: Walk me through an upload.

Web forwards it to the master, which hashes it and checks the Redis cache. On a miss it splits the image into 512 px tiles with a halo, writes the job to a Redis hash and produces one Kafka message per tile with acks=all. Workers in the filter's consumer group process the tiles in parallel, produce results and commit offsets. The master de-duplicates with ZADD NX, counts with HINCRBY, reassembles on the last tile and caches the result for 24 hours.

Q: What does the KRaft controller do?

It owns the cluster metadata: registers brokers and tracks their heartbeats, creates topics and places partitions, and elects a new partition leader from the ISR when a broker fails. It stores no messages and does not manage consumer groups.

Q: How is a new partition leader chosen?

Not by a vote: the controller picks the first replica in the replica list that is alive and in the ISR, bumps the leader epoch and records the change. Only ISR members qualify because with acks=all they hold every acknowledged message.

Q: What happens if a worker crashes mid-job?

Its Redis heartbeat key expires and the dashboard shows it dead. Independently, the group coordinator stops receiving its consumer heartbeats and rebalances its partition to the other worker, which resumes from the committed offset. A tile it was halfway through is reprocessed and the duplicate is ignored.

Q: Why commit after processing?

Commit-before is at-most-once (a crash loses the tile). Commit-after is at-least-once (a crash causes a retry); duplicates are harmless because processing is idempotent.

Q: How is a group's coordinator chosen?

It is the leader of __consumer_offsets partition abs(hash(group.id)) mod N. With N = 2 and chosen group names, the two worker groups get coordinators on different brokers.

Q: What limits parallelism?

Partitions. Within a group each partition goes to one consumer, so 2 partitions mean at most 2 active workers per group. To scale, add partitions, then workers.

Q: How is Redis made fault tolerant?

A master with two asynchronous replicas and three Sentinels. If the master dies, the Sentinels agree by quorum, promote the most up-to-date replica, and clients find it by asking Sentinel. The last few milliseconds of writes can be lost because replication is asynchronous.

Q: Why three Sentinels?

One is a single point of failure and cannot tell a dead master from its own broken network, which could create two masters. Three with a quorum of two survives one failure and avoids split brain.

Q: How did you prevent race conditions?

Single Redis commands are atomic. HINCRBY increments inside Redis and returns a different value to each caller, so exactly one sees the job complete; ZADD NX makes duplicate tiles no-ops.

Q: How does the cache work?

Key = SHA-256 of the uploaded bytes plus the filter; value = the finished JPEG with a 24-hour TTL. Repeat requests return in about 2 ms with no Kafka or worker involvement. LRU eviction applies to keys with a TTL when memory is full.

Q: Why one Dockerfile for many services?

Kafka and Redis use official images; all Python services share the same runtime, so one image holds all the code and each container runs a different entry file set in docker-compose.

Q: What would you change for production?

Three controllers and three brokers with min.insync.replicas=2, leaders spread across brokers, one worker group with partitions = workers (or a topic per filter), TLS/SASL, and persistent volumes.

Appendix A. Specifications

Area	Specification
Images	apache/kafka:4.1.0 · redis:7.4-alpine · image-pipeline:latest (python:3.12-slim)
Python packages	confluent-kafka 2.15.1 · redis 8.1.0 · flask 3.1.3 · waitress 3.0.2 · requests 2.34.2 · numpy 2.4.6 · opencv-python-headless 4.13.0.92
Kafka	1 controller (node 0, :9093) + 2 brokers (nodes 1, 2, :9092), 512 MB heap each; cluster id MkU3OEVBNTcwNTJENDM2Qk
Topics	image_tasks 2 partitions, image_results 1 partition; RF 2; min.insync.replicas 1; retention 1 h
Groups	workers-blur, workers-blackwhite, master-assemblers; __consumer_offsets 2 partitions, RF 2
Producer	acks=all, idempotence on, murmur2 partitioner
Consumer	manual commit, auto.offset.reset=earliest, cooperative-sticky, session 10 s, heartbeat 3 s, max poll interval 60 s
Redis	1 master + 2 replicas; AOF everysec; maxmemory 512 MB, volatile-lru
Sentinel	3 instances, :26379, master name mymaster, quorum 2, down-after 5 s, failover timeout 20 s
Images	512 px tiles, 25 px halo, min 1024×1024, max upload 50 MB, JPEG quality 95, preview ≤ 480 px; png, jpg, jpeg, bmp, tiff, webp
Timing	worker heartbeat 3 s / TTL 10 s · monitor every 3 s · cache and job TTL 24 h · demo delay 0.5 s per tile
Ports on the host	web 8090 → 8080 · master 5050 → 5000

Appendix B. Glossary

Term	Meaning
acks=all	Producer waits until every in-sync replica has the message.
At-least-once	Nothing is lost, but a message may be processed more than once.
Bootstrap servers	Brokers a client contacts first to download cluster metadata.
Committed offset	Next offset a consumer group will read in a partition, stored by the coordinator.
Fencing	The controller marking a broker as dead so it cannot lead or serve.
Halo	Extra border pixels sent with a tile so filters behave correctly at tile edges.
Idempotent	Doing it twice has the same effect as doing it once.
ISR	In-sync replicas: copies fully caught up with the partition leader.
KRaft	Kafka's Raft-based metadata management, replacing ZooKeeper.
Leader epoch	Counter that increases on every leader change; rejects stale leaders.
Quorum	Minimum number of Sentinels that must agree the master is down (2 here).
Rebalance	Reassigning a group's partitions when a member joins or leaves.
Split brain	Two nodes both acting as leader or master at the same time.
TTL	Time to live: Redis deletes the key automatically when it expires.